

MUST

S.1

Self-containment

1/3

All modules and components essential to replicate computational execution MUST be reachable within a single top-level research object.

Are all files and directories nested under a common root?

✓ Yes

X No

Are datasets included, linked, or referenced reachable from the code and environment specifications without crossing the boundary of a common root?

✓ Yes

X No

Reason for not meeting requirement...

Is external software included in the environment or linked as submodules within the project boundary?

✓ Yes

X No

MUST

T.1 + T.3 + T.5

Tracking

0/3

Persistent content identification MUST be recorded for all components. Provenance of all modifications MUST be recorded. Provenance records MUST include the versions of all components involved.

Are version control systems such as `Git` used for code, text, documentation, and configuration files?

✓ Yes

X No

Are version control systems such as `git-annex`, `DataLad`, or `Git` LFS used for large binary data?

✓ Yes

X No

Are the exact environment specifications used to generate a set of results included in the provenance records to link computational actions to particular environments?

✓ Yes

X No

MUST

A.1

Actionability

0/2

Research object MUST contain sufficient instructions to reproduce all computational results.

Is a `README.md` or `Makefile` included with instructions for installation and usage?

✓ Yes

X No

Is there a clear starting point for users to start reproducing results (e.g., a main script, a workflow definition, or a container image)?

✓ Yes

X No

MUST

P.1

Portability

1/3

Procedures MUST NOT depend on undocumented host environment state.

Are relative paths used in scripts, avoiding hardcoded or system-specific paths like `C:\Users\...` or `/home/user/...`?

✓ Yes

X No

Are all software dependencies included in the environment specification, rather than relying on pre-installed tools?

✓ Yes

X No

Have all assumptions about the host system's configuration been documented (e.g., specific OS versions, required system libraries, or environment variables)?

✓ Yes

X No

MUST

P.2

Portability

0/1

Computational environments MUST be explicitly specified.

Is there a clear list of system requirements and dependencies documented in the README or environment specifications?

✓ Yes

X No

MUST

P.3

Portability

0/2

Environment definitions MUST be version controlled.

Are environment specifications (e.g., Dockerfiles, `pyproject.toml`, `package.json`) included in the version control system alongside code and data?

✓ Yes

X No

Is there a process for updating environment specifications when dependencies change, and are these updates tracked in version control?

✓ Yes

X No

MUST

D.1

Distributability

1/3

All referenced modules and components MUST be persistently retrievable by others.

Are environment specifications (e.g., container digests, frozen package manifests) included to ensure others can attempt to exactly replicate the environment?

✓ Yes

X No

Are environment specifications shared in a way that others can access and use them (e.g., published container images, archived environment files)?

✓ Yes

X No

Is there documentation on how to obtain and use the environment specifications for reproduction?

✓ Yes

X No

SHOULD

T.2

Tracking

0/1

All components SHOULD be tracked using the same content-addressed version control system.

Is a common version control system (e.g., `Git`, `DVC`, `git-annex`, or `DataLad`) used across all components?

✓ Yes

X No

SHOULD

A.2

Actionability

0/1

Procedures SHOULD be specified as executable specifications.

Is the workflow tested regularly to ensure instructions remain accurate?

✓ Yes

X No

SHOULD

M.1

Modularity

0/1

Components SHOULD be organized in a modular structure.

Are raw data, processed data, code, and environment definitions separated into distinct modules?

✓ Yes

X No

SHOULD

E.1

Ephemerality

0/2

Computational results SHOULD be produced in ephemeral environments.

Is the pipeline tested in a fresh container, batch job, clean virtual machine, or cloud-based instance?

✓ Yes

X No

Does the workflow spawn disposable environments for each execution, allowing for isolation of runs?

✓ Yes

X No

SHOULD

D.2

Distributability

0/3

Environment specifications SHOULD support reproducible builds.

Are the exact environment specifications used to generate a specific set of results regularly tested to ensure they can be reconstituted as intended?

✓ Yes

X No

Are environment artifacts archived within the research object where possible (e.g., executable binaries, container images)?

✓ Yes

X No

Is there a process for updating environment specifications when dependencies change, and are these updates tracked in version control?

✓ Yes

X No

MAY

M.2

Modularity

0/5

Components MAY be included directly or linked as subdatasets.

Are external datasets or software dependencies included as submodules or linked as submodules?

✓ Yes

X No

Is the modular structure documented to clarify how components relate and can be recombined?

✓ Yes

X No

Are modular boundaries defined in a way that allows independent updates without breaking the overall workflow?

✓ Yes

X No

Is there a clear mechanism for composing modules together (e.g., `git` submodules, or container orchestration)?

✓ Yes

X No

Is there documentation or tooling to support users in understanding and navigating the modular structure?

✓ Yes

X No